

Chapter 4: Functions

Module 1: Function Introduction

- What is a Function?
 - Why We Use Functions
 - Real-World Analogy
 - Function Syntax
 - Calling (Invoking) a Function
 - Function Execution Flow
-

Module 2: Function Declaration

- Function Declaration
 - Function Naming Rules
 - Function with No Parameters
 - Function with Parameters
 - Function with Multiple Parameters
-

Module 3: Function Expression

- What is a Function Expression?
 - Anonymous Functions
 - Named Function Expressions
 - Function Declaration vs Function Expression
-

Module 4: Parameters & Arguments

- What are Parameters?
 - What are Arguments?
 - Single Parameter
 - Multiple Parameters
 - Default Parameters
 - Passing Different Data Types
 - Parameter Order
-

Module 5: Return Statement

- What is `return`?
 - Returning Values
 - Returning Expressions
 - Multiple Return Statements
 - Early Return
 - `return` VS `console.log()`
-

Module 6: Variable Scope

- Global Scope
 - Local Scope
 - Block Scope
 - Function Scope
 - Scope Chain
 - Shadowing Variables
-

Module 7: Arrow Functions (ES6)

- Introduction
 - Syntax
 - Single Parameter
 - Multiple Parameters
 - Implicit Return
 - Explicit Return
 - Arrow Function vs Regular Function
 - `this` Difference (Introduction)
-

Module 8: Callback Functions

- What is a Callback?
 - Passing Functions as Arguments
 - Callback Execution
 - Practical Examples
-

Module 9: Higher-Order Functions

- What is a Higher-Order Function?

- Functions Returning Functions
 - Functions Accepting Functions
 - Practical Examples
-

Module 10: Recursion

- What is Recursion?
 - Base Case
 - Recursive Case
 - Call Stack
 - Practical Examples (Factorial, Countdown)
-

Module 11: Advanced Function Concepts

- First-Class Functions
 - Anonymous Functions
 - Immediately Invoked Function Expression (IIFE)
 - Closures (Introduction)
 - Lexical Scope (Introduction)
-

Interview Preparation

- Function vs Method
 - Function Declaration vs Function Expression
 - Parameters vs Arguments
 - `return` vs `console.log()`
 - Global Scope vs Local Scope
 - Arrow Function vs Regular Function
 - Callback Function
 - Higher-Order Function
 - Recursion
 - Common Interview Coding Problems
-

Practice Tasks

- Basic Function Tasks
- Parameter & Argument Tasks
- Return Statement Tasks
- Scope Tasks
- Arrow Function Tasks

- Callback Tasks
- Higher-Order Function Tasks
- Recursion Tasks
- Mixed Practice Problems

This chapter takes you from basic functions to advanced function concepts used in modern JavaScript.

Module 1: Function Introduction

What is a Function?

A **function** is a reusable block of code that performs a specific task.

Instead of writing the same code multiple times, you write it once inside a function and call it whenever you need it.

Why Do We Need Functions?

Imagine you need to greet five people.

Without a function:

```
console.log("Welcome!");  
console.log("Welcome!");  
console.log("Welcome!");  
console.log("Welcome!");  
console.log("Welcome!");
```

This repeats the same code.

With a function:

```
function greet() {  
    console.log("Welcome!");  
}  
  
greet();  
greet();  
greet();  
greet();  
greet();
```

Output:

```
Welcome!  
Welcome!  
Welcome!  
Welcome!  
Welcome!
```

This is cleaner, shorter, and easier to maintain.

Real-World Analogy

Think of a **coffee machine**.

- You press a button.
- The machine follows a set of instructions.
- It gives you coffee.

You don't need to know every internal step each time—you simply use the machine.

A function works the same way.

Example:

```
makeCoffee();
```

You call the function, and it performs its task.

Advantages of Functions

- Reuse code
 - Reduce duplication
 - Make programs easier to read
 - Make debugging easier
 - Make large projects easier to manage
-

Function Syntax

```
function functionName() {  
    // code  
}
```

Example:

```
function sayHello() {  
    console.log("Hello");  
}
```

Parts of a Function

Example:

```
function sayHello() {  
    console.log("Hello");  
}
```

1. function

This is the **keyword** that tells JavaScript you're creating a function.

```
function
```

2. Function Name

```
sayHello
```

This is the name used to call the function later.

3. Parentheses ()

```
()
```

Parentheses hold **parameters** (we'll learn them in Module 4).

For now, they're empty.

4. Curly Braces {}

```
{  
  
}
```

Everything inside the braces is the function body.

Creating Your First Function

```
function greet() {  
    console.log("Welcome to JavaScript!");  
}
```

Nothing is printed yet.

Why?

Because creating a function does **not** execute it.

Calling (Invoking) a Function

To run a function, call it using its name followed by parentheses.

```
function greet() {  
    console.log("Welcome!");  
}
```

```
greet();
```

Output:

```
Welcome!
```

Calling a Function Multiple Times

```
function greet() {  
    console.log("Hello");  
}
```

```
greet();  
greet();  
greet();
```

Output:

```
Hello  
Hello  
Hello
```

One function can be used as many times as you want.

Function Execution Flow

Example:

```
console.log("Start");  
  
function greet() {  
    console.log("Hello");  
}
```

```
greet();  
  
console.log("End");
```

Output:

```
Start  
Hello  
End
```

Execution order:

1. Print "Start"
2. Create the function
3. Call `greet()`
4. Print "Hello"
5. Print "End"

Function Naming Rules

Function names should:

- Start with a letter, `_`, or `$`
- Not start with a number
- Not use JavaScript reserved keywords
- Be meaningful

Good examples:

```
function calculateTotal() {}  
function printName() {}  
function loginUser() {}
```

Bad examples:

```
function 123test() {}  
function function() {}  
function abc() {}
```

The first two are invalid, and the third is valid but not descriptive.

Common Beginner Mistakes

Mistake 1: Forgetting to Call the Function

```
function greet() {
```



```
    console.log("Hello");  
}
```

Output:

Nothing

Correct:

```
function greet() {  
    console.log("Hello");  
}  
  
greet();
```

Output:

Hello

Mistake 2: Forgetting Parentheses

Wrong:

```
greet;
```

This only refers to the function.

Correct:

```
greet();
```

This executes the function.

Mistake 3: Misspelling the Function Name

```
function greet() {  
    console.log("Hello");  
}  
  
great();
```

Output:

ReferenceError

Because `great` is different from `greet`.

Best Practices

- Give functions meaningful names.
 - Make each function perform one specific task.
 - Avoid writing very large functions.
 - Reuse functions whenever possible.
-

Interview Questions & Answers

1. What is a function in JavaScript?

Answer:

A function is a reusable block of code designed to perform a specific task. It executes only when it is called.

2. Why do we use functions?

Answer:

We use functions to:

- Reuse code
 - Reduce duplication
 - Improve readability
 - Simplify maintenance
 - Organize programs
-

3. What is the syntax of a function?

Answer:

```
function functionName() {  
  
}
```

4. What is the difference between creating and calling a function?

Answer:

Creating (declaring) a function defines it.

```
function greet() {  
  console.log("Hello");  
}
```

Calling (invoking) a function executes it.

```
greet();
```

5. What keyword is used to create a function?

Answer:

```
function
```

6. What happens if you don't call a function?

Answer:

The function is created but never executed, so its code does not run.

7. Can a function be called multiple times?

Answer:

Yes. A function can be called as many times as needed.

Example:

```
greet();  
greet();  
greet();
```

8. What are the main parts of a function?

Answer:

- function keyword
 - Function name
 - Parentheses ()
 - Function body {}
-

9. What is function invocation?

Answer:

Function invocation means calling a function to execute its code.

Example:

```
greet();
```

10. What will be the output?

```
function welcome() {  
    console.log("Welcome");  
}
```

```
welcome();  
welcome();
```

Answer:

```
Welcome  
Welcome
```

Practice Tasks

Task 1

Create a function named `sayHello` that prints:

```
Hello JavaScript
```

Call it **3 times**.

Task 2

Create a function named `showName` that prints your name.

Call it **5 times**.

Task 3

Create a function named `welcomeMessage` that prints:

```
Welcome to Visio Virex
```

Call it **2 times**.

Task 4

Create a function named `printLine` that prints:

=====

Call it **4 times**.

Task 5

Write a program that prints:

```
Program Started  
Hello  
Program Finished
```

using a function named `hello()` for the "Hello" line.

Complete these practice tasks before moving to **Module 2: Function Declaration**.

Module 2: Function Declaration

What is a Function Declaration?

A **Function Declaration** is the standard way of creating a function using the `function` keyword.

Syntax:

```
function functionName() {  
    // code  
}
```

Example:

```
function greet() {  
    console.log("Hello");  
}
```

Why Do We Use Function Declarations?

Function declarations help us:

- Organize code
- Reuse code

- Avoid repetition
 - Make programs easier to read
 - Make debugging easier
-

Function Declaration Syntax

```
function functionName(parameters) {  
    // Function Body  
}
```

Every function declaration has four parts:

1. **function** Keyword

Tells JavaScript that you're creating a function.

```
function
```

2. **Function Name**

```
greet
```

The name used to call the function.

3. **Parentheses ()**

Used to receive **parameters**.

For now they can be empty.

```
()
```

4. **Function Body {}**

Contains the code that runs when the function is called.

```
{  
  
}
```

Function with No Parameters

Example:

```
function welcome() {  
  console.log("Welcome!");  
}
```

```
welcome();
```

Output:

```
Welcome!
```

Function with One Parameter

A **parameter** is a variable that receives data when the function is called.

Example:

```
function greet(name) {  
  console.log("Hello " + name);  
}
```

```
greet("Mainul");
```

Output:

```
Hello Mainul
```

Here:

```
name
```

is the parameter.

```
"Mainul"
```

is the argument.

Function with Multiple Parameters

A function can receive more than one parameter.

Example:

```
function introduce(name, age) {
```

```
    console.log("Name: " + name);  
    console.log("Age: " + age);  
}  
  
introduce("Mainul", 20);
```

Output:

```
Name: Mainul  
Age: 20
```

Calling a Function Multiple Times

Example:

```
function greet(name) {  
    console.log("Hello " + name);  
}  
  
greet("Mainul");  
greet("Rahim");  
greet("Karim");
```

Output:

```
Hello Mainul  
Hello Rahim  
Hello Karim
```

Notice that the same function works with different values.

Function Naming Rules

A function name:

- Should start with a letter, `_`, or `$`
- Cannot start with a number
- Cannot be a JavaScript keyword
- Should describe what the function does

Good examples:

```
function calculateTotal() {}  
  
function printName() {}  
  
function loginUser() {}
```

Bad examples:


```
function 123abc() {}  
  
function function() {}  
  
function x() {}
```

Naming Convention (camelCase)

JavaScript uses **camelCase** for function names.

Example:

```
function calculatePrice() {}  
  
function getUser_name() {}  
  
function printStudentInfo() {}
```

Not:

```
calculate_price()  
  
CalculatePrice()  
  
CALCULATEPRICE()
```

Function Hoisting

One special feature of function declarations is **hoisting**.

You can call a function **before** it is declared.

Example:

```
greet();  
  
function greet() {  
    console.log("Hello");  
}
```

Output:

Hello

Why?

During execution, JavaScript moves function declarations to the top of their scope before running the code.

Conceptually, JavaScript treats it like:

```
function greet() {  
    console.log("Hello");  
}
```

```
greet();
```

We will study **Hoisting** in detail in the Advanced JavaScript chapter.

Common Beginner Mistakes

Mistake 1: Forgetting Parentheses

Wrong:

```
greet;
```

Nothing happens.

Correct:

```
greet();
```

Mistake 2: Wrong Function Name

```
function greet() {  
    console.log("Hello");  
}
```

```
great();
```

Output:

```
ReferenceError
```

Because:

```
greet
```

is different from

```
great
```

Mistake 3: Missing Curly Braces

Wrong:

```
function greet()  
  console.log("Hello");
```

Correct:

```
function greet() {  
  console.log("Hello");  
}
```

Mistake 4: Forgetting Arguments

```
function greet(name) {  
  console.log("Hello " + name);  
}
```

```
greet();
```

Output:

```
Hello undefined
```

Because no value was passed to the `name` parameter.

Best Practices

- Use meaningful function names.
 - Keep each function focused on one task.
 - Follow camelCase naming.
 - Reuse functions instead of copying code.
 - Keep function bodies short and readable.
-

Interview Questions & Answers

1. What is a function declaration?

Answer:

A function declaration is the standard way of defining a function using the `function` keyword.

Example:

```
function greet() {  
  console.log("Hello");  
}
```

2. What keyword is used to declare a function?

Answer:

`function`

3. What are the four parts of a function declaration?

Answer:

1. `function` keyword
 2. Function name
 3. Parentheses `()`
 4. Function body `{ }`
-

4. What is the difference between a parameter and an argument?

Answer:

A **parameter** is the variable listed in the function definition.

An **argument** is the actual value passed when calling the function.

Example:

```
function greet(name) {  
    console.log(name);  
}
```

```
greet("Mainul");
```

- Parameter: `name`
 - Argument: `"Mainul"`
-

5. Can a function have multiple parameters?

Answer:

Yes.

Example:

```
function add(a, b) {  
  
}
```

6. What is camelCase?

Answer:

A naming convention where the first word starts with a lowercase letter and each following word starts with an uppercase letter.

Example:

```
calculateTotal()
```

7. What is function hoisting?

Answer:

Function hoisting allows a function declaration to be called before it appears in the code because JavaScript hoists function declarations during execution.

8. What happens if you call a function without providing a required argument?

Answer:

The corresponding parameter receives `undefined`.

Example:

```
function greet(name) {  
    console.log(name);  
}  
  
greet();
```

Output:

```
undefined
```

9. Can we call a function more than once?

Answer:

Yes. A function can be called as many times as needed with different arguments.

10. What will be the output?

```
function greet(name) {
```

```
    console.log("Hello " + name);  
}
```

```
greet("Mainul");  
greet("Rahim");
```

Answer:

```
Hello Mainul  
Hello Rahim
```

Practice Tasks

Task 1

Create a function named `printMessage` that prints:

```
Learning JavaScript
```

Call it **3 times**.

Task 2

Create a function named `sayHello` that accepts one parameter `name`.

Call it with:

```
Mainul  
Rahim  
Karim
```

Expected output:

```
Hello Mainul  
Hello Rahim  
Hello Karim
```

Task 3

Create a function named `studentInfo` that accepts two parameters:

- `name`
- `age`

Call it for **two students**.

Task 4

Create a function named `multiply` that accepts two numbers and prints:

```
First Number: ...  
Second Number: ...
```

(No need to calculate the multiplication yet.)

Task 5

Call a function **before** its declaration and observe that it still works. This demonstrates **function hoisting**.

Module 3: Function Expression

What is a Function Expression?

A **Function Expression** is a function that is **stored inside a variable**.

Unlike a function declaration, the function becomes the **value of a variable**.

Syntax

```
const variableName = function () {  
    // code  
};
```

Example:

```
const greet = function () {  
    console.log("Hello");  
};  
  
greet();
```

Output:

```
Hello
```

Function Declaration vs Function Expression

Function Declaration

```
function greet() {  
  console.log("Hello");  
}  
  
greet();
```

Function Expression

```
const greet = function () {  
  console.log("Hello");  
};  
  
greet();
```

Both produce:

Hello

The difference is **how they are created**.

How Function Expressions Work

Example:

```
const welcome = function () {  
  console.log("Welcome");  
};  
  
welcome();
```

Step 1:

A function is created.

Step 2:

The function is stored inside the variable:

welcome

Step 3:

Calling

```
welcome();
```


executes the function.

Anonymous Function

An **anonymous function** is a function **without a name**.

Example:

```
const greet = function () {  
  console.log("Hello");  
};
```

Notice:

```
function ()
```

There is no function name.

Most function expressions use anonymous functions.

Named Function Expression

A function expression can also have its own name.

Example:

```
const greet = function sayHello() {  
  console.log("Hello");  
};  
  
greet();
```

Output:

```
Hello
```

Here:

Variable name:

```
greet
```

Function name:

```
sayHello
```

In practice, anonymous function expressions are much more common.

Function Expression with Parameters

Example:

```
const greet = function (name) {  
  console.log("Hello " + name);  
};  
  
greet("Mainul");
```

Output:

Hello Mainul

Function Expression with Multiple Parameters

```
const add = function (a, b) {  
  console.log(a + b);  
};  
  
add(10, 20);
```

Output:

30

Function Expression with Return

```
const multiply = function (a, b) {  
  return a * b;  
};  
  
let result = multiply(5, 4);  
  
console.log(result);
```

Output:

20

Hoisting Difference

This is one of the most important interview topics.

Function Declaration

```
greet();

function greet() {
  console.log("Hello");
}
```

Output:

Hello

Works because function declarations are hoisted.

Function Expression

```
greet();

const greet = function () {
  console.log("Hello");
};
```

Output:

ReferenceError

Why?

Because:

```
const greet
```

exists but isn't initialized until that line executes.

So you **cannot call a function expression before it is assigned.**

Function Declaration vs Function Expression

Function Declaration	Function Expression
Uses <code>function</code> keyword directly	Function stored in a variable
Has a function name	Usually anonymous
Hoisted	Not fully hoisted
Can be called before declaration	Cannot be called before assignment

Common Beginner Mistakes

Mistake 1: Calling Before Assignment

Wrong:

```
sayHi();

const sayHi = function () {
  console.log("Hi");
};
```

Output:

ReferenceError

Correct:

```
const sayHi = function () {
  console.log("Hi");
};

sayHi();
```

Mistake 2: Forgetting Semicolon

```
const greet = function () {
  console.log("Hello");
}
```

Although JavaScript often inserts it automatically, the recommended style is:

```
const greet = function () {
  console.log("Hello");
};
```

Mistake 3: Forgetting Parentheses

Wrong:

```
greet;
```

Correct:

```
greet();
```

Mistake 4: Confusing Function Name and Variable Name

```
const greet = function hello() {  
  console.log("Hello");  
};
```

```
greet();
```

Call the function using:

```
greet();
```

not:

```
hello();
```

Best Practices

- Use `const` when creating function expressions.
 - Prefer anonymous function expressions unless a name is needed.
 - Use meaningful variable names.
 - Call the function only after it has been assigned.
-

Interview Questions & Answers

1. What is a function expression?

Answer:

A function expression is a function stored inside a variable.

Example:

```
const greet = function () {  
  console.log("Hello");  
};
```

2. What is the difference between a function declaration and a function expression?

Answer:

Function declarations are hoisted and can be called before they are declared.

Function expressions are assigned to variables and cannot be called before the assignment.

3. What is an anonymous function?

Answer:

An anonymous function is a function without a name.

Example:

```
const greet = function () {};
```

4. What is a named function expression?

Answer:

A function expression that includes a function name.

Example:

```
const greet = function sayHello() {};
```

5. Are function expressions hoisted?

Answer:

No. They are not fully hoisted like function declarations.

6. Which keyword is commonly used with function expressions?

Answer:

`const`

because the function reference usually doesn't need to be reassigned.

7. What will be the output?

```
const add = function (a, b) {  
  return a + b;  
};  
  
console.log(add(5, 3));
```

Answer:

8

8. What happens if you call a function expression before it is assigned?

Answer:

It throws a `ReferenceError`.

9. What is the output?

```
const greet = function () {  
  console.log("Hello");  
};  
  
greet();
```

Answer:

Hello

10. Which is better: Function Declaration or Function Expression?

Answer:

It depends on the situation:

- **Function Declaration:** Good for reusable functions that can be called anywhere in their scope.
 - **Function Expression:** Preferred when assigning functions to variables, passing them as arguments, or creating callbacks.
-

Practice Tasks

Task 1

Create a function expression named `welcome` that prints:

```
Welcome to JavaScript
```

Call it **3 times**.

Task 2

Create a function expression named `sayHello` that accepts one parameter `name`.

Call it with:

```
Mainul  
Rahim  
Karim
```

Task 3

Create a function expression named `add` that accepts two numbers and prints their sum.

Example:

```
add(10, 20);
```

Output:

```
30
```

Task 4

Create a function expression named `square` that accepts one number and **returns** its square.

Example:

```
console.log(square(5));
```

Output:

```
25
```

Task 5

Try calling a function expression **before** it is assigned and observe the error. Then move the function call **below** the assignment and verify that it works.

Module 4: Parameters & Arguments

What are Parameters and Arguments?

Parameters and arguments are used to pass data into a function.

Without them, a function always performs the same task.

With them, a function can work with different values every time it is called.

Example:

```
function greet(name) {  
  console.log("Hello " + name);  
}  
  
greet("Mainul");  
greet("Rahim");  
greet("Karim");
```

Output:

```
Hello Mainul  
Hello Rahim  
Hello Karim
```

The same function produces different results because different values are passed to it.

What is a Parameter?

A **parameter** is a variable declared inside the parentheses of a function definition.

Example:

```
function greet(name) {  
  
}
```

Here:

name

is the **parameter**.

Think of a parameter as an **empty box** waiting for a value.

What is an Argument?

An **argument** is the actual value passed to the function when it is called.

Example:

```
greet("Mainul");
```

Here:

```
"Mainul"
```

is the **argument**.

Parameter vs Argument

Example:

```
function greet(name) {  
    console.log("Hello " + name);  
}
```

```
greet("Mainul");
```

Parameter

Argument

name

"Mainul"

Variable inside function definition Actual value passed during function call

Single Parameter

Example:

```
function printName(name) {  
    console.log(name);  
}
```

```
printName("Mainul");
```

Output:

```
Mainul
```

Multiple Parameters

A function can have multiple parameters.

Example:

```
function studentInfo(name, age) {  
    console.log("Name: " + name);  
}
```

```
    console.log("Age: " + age);  
}  
  
studentInfo("Mainul", 20);
```

Output:

```
Name: Mainul  
Age: 20
```

Parameter Order Matters

JavaScript matches arguments to parameters by **position**, not by name.

Example:

```
function introduce(name, age) {  
    console.log(name);  
    console.log(age);  
}  
  
introduce("Mainul", 20);
```

Output:

```
Mainul  
20
```

Now change the order:

```
introduce(20, "Mainul");
```

Output:

```
20  
Mainul
```

The values are assigned based on their position.

Passing Different Data Types

You can pass any JavaScript data type as an argument.

String

```
function greet(name) {  
    console.log(name);  
}
```

```
greet("Mainul");
```

Number

```
function square(num) {  
  console.log(num);  
}
```

```
square(25);
```

Boolean

```
function check(status) {  
  console.log(status);  
}
```

```
check(true);
```

Array

```
function showFruits(fruits) {  
  console.log(fruits);  
}
```

```
showFruits(["Apple", "Mango"]);
```

Object

```
function showUser(user) {  
  console.log(user);  
}
```

```
showUser({  
  name: "Mainul",  
  age: 20  
});
```

Default Parameters (ES6)

Sometimes a user doesn't provide an argument.

Default parameters allow you to set a default value.

Example:

```
function greet(name = "Guest") {  
  console.log("Hello " + name);  
}
```

```
greet();
```

Output:

```
Hello Guest
```

If a value is provided:

```
greet("Mainul");
```

Output:

```
Hello Mainul
```

Missing Arguments

Example:

```
function add(a, b) {  
    console.log(a);  
    console.log(b);  
}  
  
add(10);
```

Output:

```
10  
undefined
```

Because:

- a = 10
 - b = undefined
-

Extra Arguments

If more arguments are passed than parameters:

```
function greet(name) {  
    console.log(name);  
}  
  
greet("Mainul", 20, "Bangladesh");
```

Output:

```
Mainul
```

The extra arguments are ignored unless you explicitly access them (covered later).

Parameter Scope

Parameters are local to the function.

Example:

```
function greet(name) {  
    console.log(name);  
}
```

```
greet("Mainul");
```

```
console.log(name);
```

Output:

```
Mainul  
ReferenceError
```

name exists only inside the function.

Common Beginner Mistakes

Mistake 1: Confusing Parameter and Argument

```
function greet(name) {  
  
}
```

```
greet("Mainul");
```

- Parameter → name
 - Argument → "Mainul"
-

Mistake 2: Wrong Argument Order

```
function student(name, age) {  
    console.log(name);  
    console.log(age);  
}
```

```
student(20, "Mainul");
```

Output:

20
Mainul

Mistake 3: Forgetting an Argument

```
function multiply(a, b) {  
    console.log(a * b);  
}  
  
multiply(5);
```

Output:

NaN

Because:

5 * undefined

results in:

NaN

Mistake 4: Expecting Parameters Outside the Function

```
function greet(name) {  
    console.log(name);  
}  
  
console.log(name);
```

Output:

ReferenceError

Best Practices

- Use meaningful parameter names.
 - Keep the number of parameters reasonable.
 - Use default parameters when appropriate.
 - Pass arguments in the correct order.
-

Interview Questions & Answers

1. What is a parameter?

Answer:

A parameter is a variable listed in a function definition that receives values when the function is called.

2. What is an argument?

Answer:

An argument is the actual value passed to a function during a function call.

3. What is the difference between a parameter and an argument?

Answer:

- Parameter → Variable in the function definition.
 - Argument → Actual value passed during the function call.
-

4. Can a function have multiple parameters?

Answer:

Yes.

Example:

```
function add(a, b) {  
  }  
}
```

5. What happens if an argument is missing?

Answer:

The corresponding parameter becomes `undefined` unless a default value is provided.

6. What are default parameters?

Answer:

Default parameters assign a default value if no argument is passed.

Example:

```
function greet(name = "Guest") {  
  console.log(name);  
}
```

7. What happens if extra arguments are passed?

Answer:

They are ignored unless accessed using features like the `arguments` object or rest parameters (covered later).

8. Are parameters local or global?

Answer:

Parameters are local to the function.

9. What will be the output?

```
function add(a, b) {  
  console.log(a + b);  
}
```

```
add(10, 20);
```

Answer:

30

10. What will be the output?

```
function greet(name = "Guest") {  
  console.log(name);  
}
```

```
greet();
```

Answer:

Guest

Practice Tasks

Task 1

Create a function `greet(name)` that prints:

```
Hello Mainul
```

Call it with your name.

Task 2

Create a function `studentInfo(name, age)`.

Call it for three different students.

Task 3

Create a function `add(a, b)` that prints the sum of two numbers.

Test it with different values.

Task 4

Create a function `multiply(a, b)` that returns the product of two numbers.

Print the returned value using `console.log()`.

Task 5

Create a function `welcome(name = "Guest")`.

- Call it without an argument.
- Call it with your name.

Observe the difference in the output.

Module 5: Return Statement

What is the `return` Statement?

The `return` statement is used to **send a value back from a function** to the place where the function was called.

Think of a function as a machine:

- **Input** → Parameters
- **Process** → Function body
- **Output** → `return`

Example:

```
function add(a, b) {  
  return a + b;  
}  
  
let result = add(10, 20);  
  
console.log(result);
```

Output:

30

The function returns 30, which is stored in the variable `result`.

Why Do We Use `return`?

Without `return`, a function can perform an action (like printing), but it cannot give a value back to the rest of your program.

Example:

```
function add(a, b) {  
  console.log(a + b);  
}  
  
let result = add(10, 20);  
  
console.log(result);
```

Output:

30
undefined

Why?

- `console.log(a + b)` prints 30.
 - The function **does not return anything**, so `result` becomes undefined.
-

Syntax

```
function functionName() {  
    return value;  
}
```

Returning a Value

Example:

```
function greet() {  
    return "Hello";  
}  
  
let message = greet();  
  
console.log(message);
```

Output:

Hello

Returning a Number

```
function square(number) {  
    return number * number;  
}  
  
console.log(square(5));
```

Output:

25

Returning an Expression

The returned value can be the result of an expression.

```
function add(a, b) {  
    return a + b;  
}
```

```
}  
  
console.log(add(15, 25));
```

Output:

```
40
```

Returning a Boolean

```
function isAdult(age) {  
    return age >= 18;  
}  
  
console.log(isAdult(20));  
console.log(isAdult(15));
```

Output:

```
true  
false
```

Returning a String

```
function fullName(firstName, lastName) {  
    return firstName + " " + lastName;  
}  
  
console.log(fullName("Mainul", "Islam"));
```

Output:

```
Mainul Islam
```

Returning an Array

```
function getColors() {  
    return ["Red", "Green", "Blue"];  
}  
  
console.log(getColors());
```

Output:

```
["Red", "Green", "Blue"]
```

Returning an Object

```
function getUser() {  
  return {  
    name: "Mainul",  
    age: 20  
  };  
}
```

```
console.log(getUser());
```

Output:

```
{ name: "Mainul", age: 20 }
```

Storing the Returned Value

```
function multiply(a, b) {  
  return a * b;  
}  
  
let answer = multiply(6, 8);  
  
console.log(answer);
```

Output:

```
48
```

Using a Returned Value in Another Calculation

```
function add(a, b) {  
  return a + b;  
}  
  
let total = add(10, 20);  
  
console.log(total * 2);
```

Output:

```
60
```

Returning from Multiple Functions

```
function add(a, b) {  
  return a + b;  
}
```

```
function multiply(a, b) {  
    return a * b;  
}  
  
console.log(add(10, 20));  
console.log(multiply(10, 20));
```

Output:

```
30  
200
```

Early Return

A function stops executing immediately after a `return` statement.

Example:

```
function checkAge(age) {  
    if (age < 18) {  
        return "Not Allowed";  
    }  
  
    return "Allowed";  
}  
  
console.log(checkAge(15));  
console.log(checkAge(20));
```

Output:

```
Not Allowed  
Allowed
```

Code After `return`

Anything after `return` inside the same block **never runs**.

```
function greet() {  
    return "Hello";  
    console.log("Welcome");  
}
```

Output:

```
Hello
```

The line:

```
console.log("Welcome");
```

is unreachable.

return Without a Value

```
function test() {  
    return;  
}  
  
console.log(test());
```

Output:

undefined

return VS console.log()

This is a very common interview question.

Using console.log()

```
function add(a, b) {  
    console.log(a + b);  
}  
  
add(5, 3);
```

Output:

8

You can see the result, but you **cannot reuse it**.

Using return

```
function add(a, b) {  
    return a + b;  
}  
  
let result = add(5, 3);  
  
console.log(result);
```


Output:

8

Now the value is stored and can be used later.

`console.log()` **VS** `return`

`console.log()`

`return`

Displays output

Sends a value back

Used for debugging or displaying

Used to produce reusable results

Doesn't stop the function

Ends the function immediately

Doesn't give a value to the caller

Gives a value to the caller

Common Beginner Mistakes

Mistake 1: Using `console.log()` Instead of `return`

```
function add(a, b) {  
  console.log(a + b);  
}  
  
let result = add(2, 3);  
  
console.log(result);
```

Output:

5
undefined

Correct

```
function add(a, b) {  
  return a + b;  
}  
  
let result = add(2, 3);  
  
console.log(result);
```

Output:

5

Mistake 2: Writing Code After `return`

```
function greet() {  
    return "Hello";  
    console.log("Hi");  
}
```

The `console.log("Hi")` line never executes.

Mistake 3: Forgetting to Store the Returned Value

```
function square(num) {  
    return num * num;  
}  
  
square(5);
```

This works, but the returned value is ignored.

Better:

```
let result = square(5);  
  
console.log(result);
```

Best Practices

- Use `return` when another part of your program needs the result.
 - Avoid writing code after a `return`.
 - Return one clear value whenever possible.
 - Use `console.log()` for debugging, not as a replacement for `return`.
-

Interview Questions & Answers

1. What is the purpose of the `return` statement?

Answer:

The `return` statement sends a value back from a function to the code that called it.

2. What happens after a `return` statement executes?

Answer:

The function stops executing immediately.

3. What is the difference between `return` and `console.log()`?

Answer:

- `return` sends a value back to the caller.
 - `console.log()` only prints a value to the console.
-

4. Can a function return any data type?

Answer:

Yes. A function can return:

- Number
 - String
 - Boolean
 - Array
 - Object
 - `null`
 - `undefined`
-

5. What is an early return?

Answer:

An early return exits the function before reaching the end when a condition is met.

6. What does a function return if no `return` statement is written?

Answer:

It automatically returns:

`undefined`

7. What will be the output?

```
function add(a, b) {  
    return a + b;  
}  
  
console.log(add(10, 5));
```

Answer:

15

8. What will be the output?

```
function greet() {  
    console.log("Hello");  
}  
  
console.log(greet());
```

Answer:

Hello
undefined

9. Can a function have multiple `return` statements?

Answer:

Yes, but only **one** `return` statement executes during a single function call.

10. What is the output?

```
function test() {  
    return;  
}  
  
console.log(test());
```

Answer:

undefined

Practice Tasks

Task 1

Create a function `add(a, b)` that **returns** the sum of two numbers.

Print the returned value.

Task 2

Create a function `square(number)` that returns the square of a number.

Test it with different values.

Task 3

Create a function `isEven(number)` that returns:

- `true` if the number is even
 - `false` otherwise
-

Task 4

Create a function `fullName(firstName, lastName)` that returns the full name.

Example:

Mainul Islam

Task 5

Create a function `checkPassword(password)`.

- If the password is "admin123", return "Access Granted".
- Otherwise, return "Access Denied".

Module 6: Variable Scope

What is Variable Scope?

Variable Scope determines **where a variable can be accessed** in your program.

In simple words:

Scope defines the visibility and lifetime of a variable.

Example:

```
let name = "Mainul";  
  
console.log(name);
```

Output:

```
Mainul
```

The variable `name` can be accessed because it is in the current scope.

Why is Scope Important?

Without scope:

- Variables could accidentally overwrite each other.
- Bugs would be harder to find.
- Large applications would be difficult to manage.

Scope helps keep variables organized and prevents naming conflicts.

Types of Scope in JavaScript

There are four main types:

1. Global Scope
 2. Function Scope
 3. Block Scope
 4. Scope Chain
-

1. Global Scope

A variable declared **outside** of any function or block has **global scope**.

It can be accessed from anywhere in the program.

Example:

```
let website = "Visio Virex";

function showWebsite() {
  console.log(website);
}

console.log(website);

showWebsite();
```

Output:

```
Visio Virex
Visio Virex
```

The variable `website` is available both outside and inside the function.

Another Example

```
let age = 20;

function printAge() {
  console.log(age);
}

printAge();

console.log(age);
```

Output:

```
20
20
```

Advantages of Global Variables

- Accessible everywhere.
 - Useful for shared values (e.g., app name, configuration).
-

Disadvantages

Too many global variables can:

- Cause naming conflicts.
- Be modified accidentally.
- Make debugging difficult.

Use global variables only when necessary.

2. Function Scope

Variables declared inside a function are available **only inside that function**.

Example:

```
function greet() {  
    let message = "Hello";  
    console.log(message);  
}  
greet();
```

Output:

Hello

Now try:

```
function greet() {  
    let message = "Hello";  
}  
console.log(message);
```

Output:

ReferenceError

Because `message` exists only inside the function.

Local Variables

Variables declared inside a function are called **local variables**.

Example:

```
function student() {  
    let name = "Mainul";
```



```
    console.log(name);  
  }  
}
```

```
student();
```

Output:

```
Mainul
```

Outside the function:

```
console.log(name);
```

Output:

```
ReferenceError
```

3. Block Scope

A **block** is any code inside curly braces {}.

Variables declared with:

- `let`
- `const`

are block-scoped.

Example:

```
if (true) {  
    let city = "Dhaka";  
    console.log(city);  
}
```

Output:

```
Dhaka
```

Outside the block:

```
console.log(city);
```

Output:

```
ReferenceError
```

Example with `for`

```
for (let i = 1; i <= 3; i++) {  
  console.log(i);  
}  
  
console.log(i);
```

Output:

```
1  
2  
3  
ReferenceError
```

The variable `i` only exists inside the loop.

`var` and Block Scope

`var` is **not block-scoped**.

Example:

```
if (true) {  
  var language = "JavaScript";  
}  
  
console.log(language);
```

Output:

```
JavaScript
```

Even though `language` was declared inside the `if` block, it is still accessible outside.

This is one reason why modern JavaScript prefers `let` and `const`.

4. Scope Chain

A function first looks for a variable in its own scope.

If it doesn't find it, JavaScript looks in the outer scope.

This process continues until the global scope.

This is called the **scope chain**.

Example:

```
let country = "Bangladesh";

function outer() {
    let city = "Dhaka";

    function inner() {
        console.log(country);
        console.log(city);
    }

    inner();
}

outer();
```

Output:

```
Bangladesh
Dhaka
```

The `inner()` function can access variables from its own scope and all outer scopes.

Variable Shadowing

A local variable can have the same name as a global variable.

The local variable takes priority inside its scope.

Example:

```
let name = "Mainul";

function showName() {
    let name = "Rahim";

    console.log(name);
}

showName();

console.log(name);
```

Output:

```
Rahim  
Mainul
```

The global variable is **shadowed** by the local variable inside the function.

Nested Functions and Scope

```
let company = "Visio Virex";  
  
function outer() {  
    let department = "Development";  
  
    function inner() {  
        let role = "Frontend Developer";  
  
        console.log(company);  
        console.log(department);  
        console.log(role);  
    }  
  
    inner();  
}  
  
outer();
```

Output:

```
Visio Virex  
Development  
Frontend Developer
```

The `inner()` function can access all outer variables.

The reverse is **not** true.

Scope Summary

Scope	Accessible Where?
Global Scope	Everywhere
Function Scope	Only inside the function
Block Scope (<code>let</code> , <code>const</code>)	Only inside the block <code>{ }</code>

Scope

Accessible Where?

Scope Chain

Inner scope can access outer scope

Common Beginner Mistakes

Mistake 1: Accessing a Local Variable Outside

```
function test() {  
    let score = 100;  
}  
  
console.log(score);
```

Output:

ReferenceError

Mistake 2: Thinking `let` is Global

```
if (true) {  
    let age = 20;  
}  
  
console.log(age);
```

Output:

ReferenceError

Mistake 3: Using `var` Like `let`

```
if (true) {  
    var city = "Dhaka";  
}  
  
console.log(city);
```

Output:

Dhaka

Many beginners expect an error here, but `var` is function-scoped, not block-scoped.

Mistake 4: Shadowing Variables Accidentally

```
let count = 10;

function test() {
    let count = 20;

    console.log(count);
}

test();

console.log(count);
```

Output:

```
20
10
```

Best Practices

- Prefer `const` whenever a variable won't be reassigned.
 - Use `let` when reassignment is needed.
 - Avoid `var` in modern JavaScript.
 - Keep variables in the smallest scope possible.
 - Avoid unnecessary global variables.
-

Interview Questions & Answers

1. What is variable scope?

Answer:

Variable scope determines where a variable can be accessed in a program.

2. What is global scope?

Answer:

A variable declared outside any function or block is globally scoped and can be accessed from anywhere.

3. What is function scope?

Answer:

Variables declared inside a function can only be accessed within that function.

4. What is block scope?

Answer:

Variables declared with `let` and `const` inside `{ }` are only accessible within that block.

5. Is `var` block-scoped?

Answer:

No. `var` is function-scoped.

6. What is a local variable?

Answer:

A variable declared inside a function or block that is only accessible there.

7. What is the scope chain?

Answer:

The scope chain is JavaScript's process of looking for a variable in the current scope, then outer scopes, until the global scope.

8. What is variable shadowing?

Answer:

Variable shadowing occurs when a local variable has the same name as an outer variable, causing the local variable to take precedence within its scope.

9. What will be the output?

```
let x = 10;

function test() {
  console.log(x);
}

test();
```

Answer:

10

10. What will be the output?

```
function demo() {

  let message = "Hello";

}

console.log(message);
```

Answer:

ReferenceError

Practice Tasks

Task 1

Create a global variable `website` with your website or brand name.

Print it:

- Outside a function
 - Inside a function
-

Task 2

Create a function `student()`.

Inside it, create a local variable `name` and print it.

Then try printing `name` outside the function.

Observe the result.

Task 3

Create an `if` block.

Inside it, declare a variable using `let`.

Try accessing it outside the block.

Observe the error.

Task 4

Repeat Task 3 using `var`.

Observe the difference in behavior.

Task 5

Create a global variable `language = "JavaScript"`.

Inside a function, create another variable with the same name but value `"TypeScript"`.

Print the variable:

- Inside the function
- Outside the function

Observe **variable shadowing**.

Module 7: Arrow Functions (ES6)

What are Arrow Functions?

An **Arrow Function** is a shorter and cleaner way to write functions in JavaScript.

It was introduced in **ES6 (ECMAScript 2015)**.

Instead of using the `function` keyword, arrow functions use the `=>` (arrow) operator.

Why Do We Use Arrow Functions?

Arrow functions make code:

- Shorter
 - Cleaner
 - Easier to read
 - Common in modern JavaScript
 - Widely used in React, Node.js, and modern JavaScript projects
-

Basic Syntax

Regular Function

```
function greet() {  
  console.log("Hello");  
}
```

Arrow Function

```
const greet = () => {  
  console.log("Hello");  
};
```

Both produce:

Hello

Understanding the Syntax

```
const greet = () => {  
  console.log("Hello");  
};
```

Parts:

const

Stores the function inside a variable.

```
greet
```

Variable name.

```
()
```

Parameters.

```
=>
```

Arrow operator.

```
{  
}
```

Function body.

Arrow Function with One Parameter

Regular Function

```
function greet(name) {  
    console.log("Hello " + name);  
}
```

Arrow Function

```
const greet = (name) => {  
    console.log("Hello " + name);  
};  
  
greet("Mainul");
```

Output

```
Hello Mainul
```

Omitting Parentheses

If there is **only one parameter**, parentheses are optional.

Both are correct:

```
const greet = (name) => {  
  console.log(name);  
};  
const greet = name => {  
  console.log(name);  
};
```

Multiple Parameters

When there are multiple parameters, parentheses are required.

```
const add = (a, b) => {  
  console.log(a + b);  
};  
  
add(10, 20);
```

Output

30

No Parameters

Use empty parentheses.

```
const welcome = () => {  
  console.log("Welcome");  
};  
  
welcome();
```

Output

Welcome

Arrow Function with Return

Regular Function

```
function square(num) {  
  return num * num;  
}
```

Arrow Function

```
const square = (num) => {
```

```
    return num * num;
};

console.log(square(5));
```

Output

25

Implicit Return

If the function has only **one expression**, you can remove:

- Curly braces { }
- return keyword

Example:

```
const square = num => num * num;

console.log(square(5));
```

Output

25

This is called **Implicit Return**.

JavaScript automatically returns the result.

Explicit Return

When using curly braces, you must write `return`.

```
const square = (num) => {
    return num * num;
};
```

This is called **Explicit Return**.

Returning an Object

This is a common interview question.

Wrong:

```
const getUser = () => {  
  name: "Mainul";  
};
```

Output

undefined

JavaScript thinks {} is the function body.

Correct:

```
const getUser = () => ({  
  name: "Mainul",  
  age: 20  
});  
  
console.log(getUser());
```

Output

```
{  
  name: "Mainul",  
  age: 20  
}
```

Wrap the object in parentheses ().

Arrow Function vs Regular Function

Regular Function

```
function add(a, b) {  
  return a + b;  
}
```

Arrow Function

```
const add = (a, b) => {  
  return a + b;  
};
```

Short Arrow Function

```
const add = (a, b) => a + b;
```

All three return the same result.

`this` in Arrow Functions

This is one of the most important interview topics.

Regular Function

A regular function has its own `this` value.

Arrow Function

An arrow function **does not create its own `this`**.

Instead, it uses the `this` value from its surrounding (lexical) scope.

Example (Introduction)

```
const person = {
  name: "Mainul",

  greet: function () {
    console.log(this.name);
  }
};

person.greet();
```

Output

Mainul

We'll study `this` in detail in the Advanced JavaScript chapter.

When Should You Use Arrow Functions?

Use arrow functions for:

- Small utility functions
 - Callback functions
 - Array methods (`map`, `filter`, `forEach`, etc.)
 - Modern JavaScript development
-

When Should You Avoid Arrow Functions?

Avoid arrow functions:

- When creating object methods that rely on their own `this`
 - When you need the `arguments` object
 - When using constructors with `new` (arrow functions cannot be constructors)
-

Common Beginner Mistakes

Mistake 1: Forgetting `return`

Wrong

```
const add = (a, b) => {  
  a + b;  
};
```

Output

undefined

Correct

```
const add = (a, b) => {  
  return a + b;  
};
```

Or

```
const add = (a, b) => a + b;
```

Mistake 2: Returning an Object Incorrectly

Wrong

```
const user = () => {  
  name: "Mainul";  
};
```

Correct

```
const user = () => ({  
  name: "Mainul"  
});
```

Mistake 3: Forgetting Parentheses for Multiple Parameters

Wrong

```
const add = a, b => a + b;
```

Correct

```
const add = (a, b) => a + b;
```

Mistake 4: Expecting Arrow Functions to Have Their Own `this`

Arrow functions inherit `this` from the surrounding scope.

Best Practices

- Use `const` when creating arrow functions.
 - Use implicit return for simple one-line expressions.
 - Use explicit return for multiple statements.
 - Use meaningful function names.
 - Prefer arrow functions for callbacks and utility functions.
-

Interview Questions & Answers

1. What is an arrow function?

Answer:

An arrow function is a shorter syntax for writing functions introduced in ES6 using the `=>` operator.

2. What are the advantages of arrow functions?

Answer:

- Shorter syntax
 - Cleaner code
 - Lexical `this`
 - Common in modern JavaScript frameworks
-

3. What is implicit return?

Answer:

If an arrow function has only one expression, JavaScript automatically returns its value.

Example:

```
const square = x => x * x;
```

4. What is explicit return?

Answer:

When using curly braces {}, you must use the `return` keyword.

5. Can arrow functions have multiple parameters?

Answer:

Yes.

Example:

```
const add = (a, b) => a + b;
```

6. Can arrow functions be called before they are defined?

Answer:

No.

They are usually assigned to variables (`const` or `let`) and are **not hoisted** like function declarations.

7. What is the main difference between a regular function and an arrow function?

Answer:

A regular function has its own `this`.

An arrow function inherits `this` from its surrounding scope.

8. Can arrow functions be used as constructors?

Answer:

No.

Arrow functions cannot be used with the `new` keyword.

9. What will be the output?

```
const multiply = (a, b) => a * b;  
console.log(multiply(4, 5));
```

Answer:

20

10. How do you return an object from an arrow function?

Answer:

Wrap the object in parentheses.

```
const getUser = () => ({  
  name: "Mainul"  
});
```

Practice Tasks

Task 1

Convert this regular function into an arrow function.

```
function greet() {  
  console.log("Hello");  
}
```

Task 2

Create an arrow function `square(num)` that returns the square of a number.

Task 3

Create an arrow function `add(a, b)` that returns the sum of two numbers using **implicit return**.

Task 4

Create an arrow function `student(name, age)` that prints:

```
Name: Mainul  
Age: 20
```

Use different values when calling the function.

Task 5

Create an arrow function `getCourse()` that returns this object:

```
{  
  title: "JavaScript",  
  duration: "3 Months"  
}
```

Then print the returned object.

Module 8: Callback Functions

What is a Callback Function?

A **callback function** is a function that is **passed as an argument to another function** and is executed later.

In simple words:

A callback is a function that another function "calls back" when it needs it.

Why Do We Use Callback Functions?

Callbacks help us:

- Reuse code
- Make functions more flexible
- Execute code after another task finishes
- Handle asynchronous operations (we'll learn this later)

Callbacks are used everywhere in JavaScript, including:

- Event handling
 - Timers (`setTimeout`, `setInterval`)
 - Array methods (`forEach`, `map`, `filter`)
 - APIs
 - Promises and Async/Await (later chapters)
-

Real-Life Analogy

Imagine you order food online.

1. You place the order.
2. The restaurant prepares the food.
3. When it's ready, the delivery person calls you.

That final call is like a **callback**.

Basic Example

```
function greet(name) {  
  console.log("Hello " + name);  
}  
  
function processUser(callback) {  
  callback("Mainul");  
}  
  
processUser(greet);
```

Output

```
Hello Mainul
```

Understanding the Example

```
function greet(name) {  
  console.log("Hello " + name);  
}
```

This is a normal function.

```
processUser(greet);
```

Notice:

We write

```
greet
```

NOT

```
greet()
```

Why?

Because:

```
greet()
```

executes the function immediately.

While

```
greet
```

passes the function itself.

Inside:

```
function processUser(callback) {  
    callback("Mainul");  
}
```

The parameter

```
callback
```

stores the function.

Then:

```
callback("Mainul");
```

calls it.

Passing Anonymous Callback

Instead of creating a separate function, we can pass one directly.

```
function processUser(callback) {
```

```
    callback();  
}  
  
processUser(function () {  
    console.log("User Processed");  
});
```

Output

User Processed

Using Arrow Functions as Callbacks

```
function processUser(callback) {  
    callback();  
}  
  
processUser(() => {  
    console.log("User Processed");  
});
```

Output

User Processed

This is the most common style in modern JavaScript.

Callback with Parameters

```
function calculate(a, b, callback) {  
    callback(a, b);  
}  
  
function add(x, y) {  
    console.log(x + y);  
}  
  
calculate(10, 20, add);
```

Output

30

Multiple Callback Functions

```
function execute(callback) {  
    callback();  
}
```

```
function first() {  
    console.log("First");  
}  
  
function second() {  
    console.log("Second");  
}  
  
execute(first);  
execute(second);
```

Output

First
Second

Callback Returning Values

```
function calculate(a, b, callback) {  
    return callback(a, b);  
}  
  
const result = calculate(10, 5, function (x, y) {  
    return x * y;  
});  
  
console.log(result);
```

Output

50

Callback with Arrow Function

```
function calculate(a, b, callback) {  
    return callback(a, b);  
}  
  
const result = calculate(5, 4, (x, y) => x + y);  
  
console.log(result);
```

Output

9

Callback Execution Flow

Example


```
function first(callback) {  
    console.log("Step 1");  
    callback();  
}  
  
function second() {  
    console.log("Step 2");  
}  
  
first(second);
```

Output

Step 1
Step 2

Execution order:

1. Call `first(second)`
2. Print "Step 1"
3. Execute `callback()`
4. `callback` is `second`
5. Print "Step 2"

Callback in `setTimeout()`

One of the most common callback examples.

```
setTimeout(function () {  
    console.log("Hello");  
}, 2000);
```

After 2 seconds:

Hello

Using an arrow function:

```
setTimeout(() => {  
    console.log("Hello");  
}, 2000);
```

Why Don't We Write `callback()` While Passing?

Wrong

```
calculate(add());
```

Correct

```
calculate(add);
```

Reason:

`add()` runs immediately.

`add` passes the function so it can be called later.

Function as a First-Class Citizen

JavaScript treats functions like values.

A function can be:

- Stored in a variable
- Passed as an argument
- Returned from another function

This is why callbacks are possible.

Example

```
const greet = function () {  
  console.log("Hello");  
};
```

The function is treated like a value.

Common Beginner Mistakes

Mistake 1: Calling Instead of Passing

Wrong

```
processUser(greet());
```

Correct

```
processUser(greet);
```

Mistake 2: Forgetting Parameters

Wrong

```
function add(a, b) {  
  console.log(a + b);  
}  
  
calculate(add);
```

If `calculate()` doesn't pass the required arguments, `a` and `b` will be `undefined`.

Mistake 3: Confusing Callback and Return

Callbacks are **functions passed into other functions**.

They are not the same as `return`.

Mistake 4: Passing a Non-Function

Wrong

```
processUser("Hello");
```

If the code tries to call it:

```
callback();
```

Output

```
TypeError
```

Because `"Hello"` is not a function.

Best Practices

- Pass the function reference, not the function call.
 - Use arrow functions for short callbacks.
 - Give callback parameters meaningful names.
 - Keep callback functions focused on one task.
-

Interview Questions & Answers

1. What is a callback function?

Answer:

A callback function is a function passed as an argument to another function, which is executed later.

2. Why are callback functions used?

Answer:

They allow functions to be more flexible and reusable, and they are commonly used for asynchronous operations and event handling.

3. What is the difference between `greet` and `greet()`?

Answer:

- `greet` → Passes the function.
 - `greet()` → Executes the function immediately.
-

4. Can arrow functions be used as callbacks?

Answer:

Yes. Arrow functions are commonly used as callbacks.

Example:

```
array.forEach(item => console.log(item));
```

5. What is the output?

```
function hello() {  
    console.log("Hello");  
}  
  
function execute(callback) {  
    callback();  
}  
  
execute(hello);
```

Answer:

Hello

6. Can a callback return a value?

Answer:

Yes. The callback can return a value, and the outer function can use or return it.

7. What happens if you pass `hello()` instead of `hello`?

Answer:

`hello()` executes immediately, and its return value is passed instead of the function itself.

8. What type of value is a callback?

Answer:

A callback is a **function**.

9. Where are callback functions commonly used?

Answer:

- `setTimeout()`
 - `setInterval()`
 - Event listeners
 - Array methods (`map`, `filter`, `forEach`)
 - Promises
 - Async/Await
-

10. What will be the output?

```
function test(callback) {  
  callback();  
}  
  
test(() => console.log("JavaScript"));
```

Answer:

JavaScript

Practice Tasks

Task 1

Create a function `execute(callback)` that calls the callback.

Create another function `sayHello()` that prints:

Hello World

Pass `sayHello` to `execute()`.

Task 2

Create a function `calculate(a, b, callback)`.

Create two callback functions:

- `add`
- `multiply`

Call `calculate()` using both callbacks.

Task 3

Call `setTimeout()` with an arrow function that prints:

Learning Callback Functions

after **3 seconds**.

Task 4

Create a function `showMessage(callback)`.

Pass an anonymous function that prints:

Welcome to Visio Virex

Task 5

Create a function `operate(a, b, callback)` that returns the callback's result.

Use an arrow function callback to subtract the two numbers and print the returned value.

Module 9: Higher-Order Functions

What is a Higher-Order Function?

A **Higher-Order Function (HOF)** is a function that does **at least one** of the following:

1. **Accepts another function as an argument**, or
2. **Returns another function**

If a function does either of these, it is called a **Higher-Order Function**.

Why Do We Use Higher-Order Functions?

Higher-order functions help us:

- Write reusable code
- Reduce code duplication
- Make programs more flexible
- Follow clean coding principles
- Build modern JavaScript applications

Many built-in JavaScript methods are Higher-Order Functions, such as:

- `forEach()`
 - `map()`
 - `filter()`
 - `find()`
 - `reduce()`
 - `sort()`
-

Relationship Between Callback and Higher-Order Function

This is a common interview question.

Example:

```
function execute(callback) {  
    callback();  
}  
  
function greet() {  
    console.log("Hello");  
}  
  
execute(greet);
```

Here:

- `execute()` → Higher-Order Function (accepts another function)
- `greet()` → Callback Function (passed into another function)

Type 1: Function Accepting Another Function

Example:

```
function calculate(a, b, operation) {  
    return operation(a, b);  
}  
  
function add(x, y) {  
    return x + y;  
}  
  
console.log(calculate(10, 20, add));
```

Output:

30

Using Arrow Functions

```
function calculate(a, b, operation) {  
    return operation(a, b);  
}  
  
const result = calculate(20, 5, (x, y) => x - y);
```



```
console.log(result);
```

Output:

15

Type 2: Function Returning Another Function

A Higher-Order Function can also return another function.

Example:

```
function greeting(message) {  
  return function(name) {  
    return message + " " + name;  
  };  
}  
  
const sayHello = greeting("Hello");  
  
console.log(sayHello("Mainul"));
```

Output:

Hello Mainul

Understanding the Flow

Step 1

```
const sayHello = greeting("Hello");
```

Returns:

```
function(name) {  
  return message + " " + name;  
}
```

Now:

sayHello

stores a function.

Step 2

```
sayHello("Mainul");
```

Output:

Hello Mainul

Returning an Arrow Function

```
const multiply = (a) => {  
  return (b) => {  
    return a * b;  
  };  
};  
  
const double = multiply(2);  
  
console.log(double(5));
```

Output:

10

Practical Example

```
function repeat(times, action) {  
  for (let i = 1; i <= times; i++) {  
    action(i);  
  }  
}  
  
repeat(3, function(number) {  
  console.log("Round:", number);  
});
```

Output:

Round: 1
Round: 2
Round: 3

Built-in Higher-Order Functions

forEach()

```
const fruits = ["Apple", "Mango", "Orange"];  
  
fruits.forEach(function(fruit) {  
  console.log(fruit);  
});
```

Output

Apple
Mango
Orange

map()

Creates a **new array**.

```
const numbers = [1, 2, 3];  
  
const doubled = numbers.map(num => num * 2);  
  
console.log(doubled);
```

Output

[2, 4, 6]

filter()

Returns matching elements.

```
const numbers = [10, 15, 20, 25];  
  
const even = numbers.filter(num => num % 2 === 0);  
  
console.log(even);
```

Output

[10, 20]

find()

Returns the **first matching element**.

```
const users = [18, 22, 15, 30];  
  
const adult = users.find(age => age >= 18);  
  
console.log(adult);
```

Output

18

reduce() (Introduction)

```
const numbers = [1, 2, 3, 4];

const sum = numbers.reduce((total, num) => total + num, 0);

console.log(sum);
```

Output

10

We'll study `reduce()` in detail in the Arrays chapter.

Higher-Order Function vs Normal Function

Normal Function

```
function add(a, b) {
  return a + b;
}
```

Doesn't accept or return another function.

Higher-Order Function

```
function calculate(a, b, operation) {
  return operation(a, b);
}
```

Accepts another function.

Common Beginner Mistakes

Mistake 1: Calling Instead of Passing

Wrong

```
calculate(10, 20, add());
```

Correct

```
calculate(10, 20, add);
```

Mistake 2: Forgetting to Return a Function

Wrong

```
function test() {  
  console.log("Hello");  
}
```

Correct

```
function test() {  
  return function() {  
    console.log("Hello");  
  };  
}
```

Mistake 3: Confusing Callback and Higher-Order Function

- Higher-Order Function → Accepts or returns functions.
 - Callback → Function passed into another function.
-

Best Practices

- Keep Higher-Order Functions simple.
 - Use meaningful callback names.
 - Prefer arrow functions for short callbacks.
 - Use built-in Higher-Order Functions (`map`, `filter`, etc.) instead of manual loops when appropriate.
-

Interview Questions & Answers

1. What is a Higher-Order Function?

Answer:

A Higher-Order Function is a function that accepts another function as an argument or returns another function.

2. Is every callback a Higher-Order Function?

Answer:

No.

A callback is the function being passed.

The function receiving it is the Higher-Order Function.

3. Name some built-in Higher-Order Functions.

Answer:

- `forEach()`
 - `map()`
 - `filter()`
 - `find()`
 - `reduce()`
-

4. Can a function return another function?

Answer:

Yes.

Functions are first-class objects in JavaScript.

5. What is the output?

```
function calculate(a, b, callback) {  
  return callback(a, b);  
}  
  
console.log(calculate(5, 3, (x, y) => x + y));
```

Answer:

8

6. What does `map()` return?

Answer:

A new array with transformed elements.

7. What does `filter()` return?

Answer:

A **new array** containing only the elements that satisfy the condition.

8. What does `find()` return?

Answer:

The **first element** that matches the condition, or `undefined` if none is found.

9. What is the difference between `map()` and `forEach()`?

Answer:

- `map()` returns a **new array**.
 - `forEach()` executes a function for each element and returns `undefined`.
-

10. Why are Higher-Order Functions important?

Answer:

They make code more reusable, flexible, readable, and are widely used in modern JavaScript frameworks like React.

Practice Tasks

Task 1

Create a Higher-Order Function `calculate(a, b, operation)`.

Create callback functions:

- `add`
- `subtract`

Use both callbacks.

Task 2

Create a function `greeting(message)` that **returns another function**.

The returned function should greet a user by name.

Example output:

```
Hello Mainul
```

Task 3

Create an array of numbers:

```
[2, 4, 6, 8, 10]
```

Use `forEach()` to print each number.

Task 4

Create an array:

```
[1, 2, 3, 4, 5]
```

Use `map()` to create a new array containing the square of each number.

Task 5

Create an array:

```
[12, 15, 18, 21, 24]
```

- Use `filter()` to create a new array of even numbers.
- Use `find()` to get the first number greater than 20.

Module 10: Recursion

What is Recursion?

Recursion is a programming technique where a **function calls itself** to solve a problem.

Instead of using a loop (`for` or `while`), the function repeats its own execution until a stopping condition is met.

Real-Life Analogy

Imagine you are standing in front of two mirrors facing each other.

Each mirror reflects the other again and again.

A recursive function behaves similarly—it keeps calling itself until it reaches a condition to stop.

Another example:

Opening nested folders:

```
Main Folder
├── Images
├── Documents
│   └── JavaScript
│       ├── Notes.pdf
│       └── Practice.js
```

To find a file, you keep opening folders inside folders. This is similar to recursion.

Why Do We Use Recursion?

Recursion is useful for problems that can be broken into **smaller versions of the same problem**, such as:

- Factorial
 - Fibonacci sequence
 - Tree traversal
 - File system navigation
 - Searching nested objects
 - DOM traversal
-

Two Parts of Every Recursive Function

Every recursive function must have:

1. Base Case

The condition that stops the recursion.

Without a base case, the function will keep calling itself forever.

2. Recursive Case

The part where the function calls itself with a smaller or simpler problem.

Basic Syntax

```
function functionName() {  
    if (baseCondition) {  
        return;  
    }  
  
    functionName(smallerProblem);  
}
```

Example 1: Countdown

```
function countdown(number) {  
    if (number === 0) {  
        console.log("Finished");  
        return;  
    }  
  
    console.log(number);  
    countdown(number - 1);  
}  
  
countdown(5);
```

Output

```
5  
4  
3  
2  
1  
Finished
```

Understanding the Flow

```
countdown(5)  
↓
```

```
countdown(4)
  ↓
countdown(3)
  ↓
countdown(2)
  ↓
countdown(1)
  ↓
countdown(0)
```

At 0, the base case is reached, and the recursion stops.

Example 2: Factorial

The factorial of a number is:

$5! = 5 \times 4 \times 3 \times 2 \times 1$

Recursive Solution

```
function factorial(n) {
    if (n === 1) {
        return 1;
    }

    return n * factorial(n - 1);
}

console.log(factorial(5));
```

Output

120

Step-by-Step Execution

```
factorial(5)
  ↓
5 × factorial(4)
  ↓
5 × 4 × factorial(3)
  ↓
5 × 4 × 3 × factorial(2)
```

↓
5 × 4 × 3 × 2 × factorial(1)
↓
5 × 4 × 3 × 2 × 1
↓
120

Example 3: Sum of Numbers

```
function sum(n) {  
    if (n === 1) {  
        return 1;  
    }  
  
    return n + sum(n - 1);  
}  
  
console.log(sum(5));
```

Output

15

Because:

5 + 4 + 3 + 2 + 1 = 15

Example 4: Power

```
function power(base, exponent) {  
    if (exponent === 0) {  
        return 1;  
    }  
  
    return base * power(base, exponent - 1);  
}  
  
console.log(power(2, 3));
```

Output

8

Because:

$2 \times 2 \times 2 = 8$

What Happens Without a Base Case?

```
function test() {  
  test();  
}
```

`test();`

Output

`RangeError: Maximum call stack size exceeded`

The function never stops calling itself.

The Call Stack

Every function call is stored in a memory structure called the **Call Stack**.

Example:

```
function one() {  
  two();  
}  
  
function two() {  
  three();  
}  
  
function three() {  
  console.log("Done");  
}  
  
one();
```

Execution order:

Call Stack

`one()`

↓

`two()`

↓

`three()`

↓
Done

↓
three() removed

↓
two() removed

↓
one() removed

Recursive functions also use the call stack.

Recursion vs Loop

Using for

```
for (let i = 5; i >= 1; i--) {  
  console.log(i);  
}
```

Using Recursion

```
function countdown(n) {  
  if (n === 0) return;  
  console.log(n);  
  countdown(n - 1);  
}  
  
countdown(5);
```

Both produce:

```
5  
4  
3  
2  
1
```

Advantages of Recursion

- Elegant solution for complex problems.
 - Great for nested data structures.
 - Easier to understand certain algorithms.
 - Often cleaner than loops for recursive problems.
-

Disadvantages

- Uses more memory due to the call stack.
 - Can be slower than loops.
 - Missing a base case causes infinite recursion.
 - Deep recursion can cause a stack overflow.
-

Common Beginner Mistakes

Mistake 1: No Base Case

```
function hello() {  
    hello();  
}
```

Results in:

Maximum call stack size exceeded

Mistake 2: Base Case Never Reached

```
function count(n) {  
  
    if (n === 0)  
        return;  
  
    count(n + 1);  
}
```

The value keeps increasing, so `n === 0` is never reached.

Mistake 3: Forgetting `return`

Wrong

```
function factorial(n) {  
    if (n === 1)  
        return 1;  
  
    n * factorial(n - 1);  
}
```

Correct

```
function factorial(n) {  
    if (n === 1)  
        return 1;  
  
    return n * factorial(n - 1);  
}
```

Best Practices

- Always define a base case.
 - Make sure each recursive call moves closer to the base case.
 - Keep recursive functions simple.
 - Use loops for simple repetition.
 - Use recursion for problems that naturally fit recursive thinking.
-

Interview Questions & Answers

1. What is recursion?

Answer:

Recursion is a technique where a function calls itself to solve a smaller version of the same problem.

2. What are the two essential parts of recursion?

Answer:

- Base Case
 - Recursive Case
-

3. What happens if there is no base case?

Answer:

The function keeps calling itself until the call stack overflows, causing:

```
RangeError: Maximum call stack size exceeded
```

4. What is the call stack?

Answer:

The call stack is a data structure that stores active function calls until they finish executing.

5. Can recursion replace loops?

Answer:

Yes, many looping problems can be solved using recursion, although loops are often more efficient for simple tasks.

6. What is the output?

```
function countdown(n) {  
    if (n === 0) {  
        return;  
    }  
  
    console.log(n);  
  
    countdown(n - 1);  
}  
  
countdown(3);
```

Answer:

```
3  
2  
1
```

7. What is the output?

```
function factorial(n) {
```

```
    if (n === 1)
        return 1;

    return n * factorial(n - 1);
}

console.log(factorial(4));
```

Answer:

24

8. Why can recursion be slower than loops?

Answer:

Because each recursive call adds a new frame to the call stack, increasing memory usage and function call overhead.

9. What types of problems are best solved with recursion?

Answer:

Problems involving:

- Trees
 - Graphs
 - Nested data
 - Factorials
 - Fibonacci sequence
 - File system traversal
-

10. What is the most common mistake beginners make with recursion?

Answer:

Forgetting the base case or writing a recursive call that never reaches the base case.

Practice Tasks

Task 1

Create a recursive function `countdown(n)` that prints numbers from `n` to 1.

Task 2

Create a recursive function `sum(n)` that returns the sum of numbers from 1 to `n`.

Example:

```
sum(5);
```

Output:

```
15
```

Task 3

Create a recursive function `factorial(n)`.

Test it with:

```
factorial(4);  
factorial(6);
```

Task 4

Create a recursive function `power(base, exponent)`.

Example:

```
power(3, 4);
```

Output:

```
81
```

Task 5

Create a recursive function `reverseCountdown(n)` that prints:

```
Finished  
1  
2  
3  
4  
5
```

when called as:

```
reverseCountdown(5);
```

Hint: Make the recursive call **before** `console.log(n)` so the numbers print while the function calls are returning from the call stack.

Module 11: Advanced Function Concepts

Introduction

In this module, you'll learn advanced function concepts that are widely used in modern JavaScript and frequently asked in interviews.

Topics covered:

1. Function Hoisting
2. Closures
3. IIFE (Immediately Invoked Function Expression)
4. `arguments` Object
5. Rest Parameters (`...`)
6. Spread Operator (`...`)
7. Default Parameters (Advanced)
8. Pure vs Impure Functions
9. First-Class Functions
10. Function Methods (`call`, `apply`, `bind`) - Introduction

1. Function Hoisting

What is Hoisting?

Hoisting is JavaScript's behavior of moving **declarations** to the top of their scope before execution.

Function Declaration

Function declarations are fully hoisted.

```
sayHello();  
  
function sayHello() {  
    console.log("Hello");  
}
```

```
}
```

Output:

```
Hello
```

Even though the function is called before it is defined, it works because function declarations are hoisted.

Function Expression

Function expressions are **not** hoisted in the same way.

```
sayHello();  
  
const sayHello = function () {  
  console.log("Hello");  
};
```

Output:

```
ReferenceError
```

Because `sayHello` is a `const` variable that isn't initialized yet.

Arrow Function

```
greet();  
  
const greet = () => {  
  console.log("Hi");  
};
```

Output:

```
ReferenceError
```

Arrow functions are also not callable before their declaration.

2. Closures

What is a Closure?

A **closure** is created when an inner function remembers and can access variables from its outer function, even after the outer function has finished executing.

Example

```
function outer() {  
    let count = 0;  
  
    return function () {  
        count++;  
        console.log(count);  
    };  
}  
  
const counter = outer();  
  
counter();  
counter();  
counter();
```

Output:

```
1  
2  
3
```

Although `outer()` has finished executing, the inner function still remembers the `count` variable.

This is called a **closure**.

Real-Life Analogy

Imagine you lock important documents in a safe and give someone the key.

Even after you leave, the person with the key can still access those documents.

The inner function is like the person with the key—it remembers the outer variables.

3. IIFE (Immediately Invoked Function Expression)

What is an IIFE?

An IIFE is a function that executes immediately after it is created.

Syntax:

```
(function () {  
    console.log("Executed");  
})();
```

Output:

Executed

Arrow Function IIFE

```
((() => {  
    console.log("Hello");  
}))();
```

Output:

Hello

Why Use an IIFE?

- Create a private scope.
 - Avoid polluting the global scope.
 - Execute setup code only once.
-

4. `arguments` Object

The `arguments` object contains all arguments passed to a regular function.

```
function show() {  
    console.log(arguments);  
}
```

```
show(10, 20, 30);
```

Output:

Arguments(3) [10, 20, 30]

Access Individual Arguments

```
function show() {  
    console.log(arguments[0]);  
    console.log(arguments[1]);  
}
```

```
show("JavaScript", "ES6");
```

Output:

JavaScript
ES6

Note: Arrow functions do **not** have their own `arguments` object.

5. Rest Parameters (...)

Rest parameters collect multiple arguments into an array.

```
function sum(...numbers) {  
  console.log(numbers);  
}
```

```
sum(10, 20, 30, 40);
```

Output:

```
[10, 20, 30, 40]
```

Example

```
function sum(...numbers) {  
  let total = 0;  
  for (const number of numbers) {  
    total += number;  
  }  
  return total;  
}  
  
console.log(sum(5, 10, 15));
```

Output:

```
30
```

Rest Parameters vs `arguments`

Rest Parameters	<code>arguments</code>
Real array	Array-like object
Works with arrow functions	Doesn't exist in arrow functions
Modern approach	Older approach

6. Spread Operator (...)

The spread operator expands arrays or objects.

Array Example

```
const numbers = [1, 2, 3];  
console.log(...numbers);
```

Output:

```
1 2 3
```

Copy an Array

```
const arr1 = [1, 2, 3];  
const arr2 = [...arr1];  
console.log(arr2);
```

Output:

```
[1, 2, 3]
```

Merge Arrays

```
const first = [1, 2];  
const second = [3, 4];  
const all = [...first, ...second];  
console.log(all);
```

Output:

```
[1, 2, 3, 4]
```

7. Default Parameters (Advanced)

```
function greet(name = "Guest") {  
  return "Hello " + name;  
}
```

```
console.log(greet());
```

Output:

```
Hello Guest
```

Default values can also depend on previous parameters.

```
function multiply(a, b = a) {  
  return a * b;  
}
```

```
console.log(multiply(5));
```

Output:

```
25
```

8. Pure vs Impure Functions

Pure Function

A pure function:

- Always gives the same output for the same input.
- Doesn't modify external data.

```
function add(a, b) {  
  return a + b;  
}
```

Impure Function

```
let total = 0;  
  
function add(value) {  
  total += value;  
}
```

This changes an external variable, so it is impure.

9. First-Class Functions

JavaScript treats functions as values.

Functions can:

- Be assigned to variables.
- Be passed as arguments.
- Be returned from functions.
- Be stored in arrays or objects.

```
const greet = function () {  
    console.log("Hello");  
};  
  
greet();
```

10. Function Methods (`call`, `apply`, `bind`) - Introduction

These methods let you control the value of `this`.

`call()`

```
const person = {  
    name: "Mainul"  
};  
  
function greet() {  
    console.log(this.name);  
}  
  
greet.call(person);
```

Output:

Mainul

`apply()`

Works like `call()`, but arguments are passed as an array.

```
function add(a, b) {  
    console.log(a + b);  
}  
  
add.apply(null, [10, 20]);
```

Output:

30

`bind()`

Returns a new function with a fixed `this`.

```
const person = {
  name: "Mainul"
};

function greet() {
  console.log(this.name);
}

const newFunction = greet.bind(person);

newFunction();
```

Output:

Mainul

We'll study these methods in depth in the **Advanced JavaScript** chapter.

Common Beginner Mistakes

Mistake 1: Calling Arrow Functions Before Declaration

```
hello();

const hello = () => {};
```

Output:

ReferenceError

Mistake 2: Forgetting Closures Remember Variables

Closures keep references to outer variables, not copies.

Mistake 3: Using `arguments` Inside Arrow Functions

```
const test = () => {
  console.log(arguments);
};
```

Output:

ReferenceError

Use rest parameters instead.

Mistake 4: Confusing Rest and Spread

```
function test(...numbers) {}
```

... collects values (Rest).

```
console.log(...numbers);
```

... expands values (Spread).

Best Practices

- Prefer **rest parameters** over `arguments`.
 - Use **spread** for copying and merging arrays.
 - Keep functions **pure** whenever possible.
 - Use **closures** when data privacy is needed.
 - Avoid unnecessary global variables.
-

Interview Questions & Answers

1. What is function hoisting?

Answer:

Function hoisting allows function declarations to be called before they appear in the code because JavaScript hoists their declarations during the creation phase.

2. What is a closure?

Answer:

A closure is a function that remembers and can access variables from its outer lexical scope even after the outer function has finished executing.

3. What is an IIFE?

Answer:

An IIFE (Immediately Invoked Function Expression) is a function that runs immediately after it is defined.

4. What is the difference between rest parameters and the spread operator?

Answer:

- **Rest** (...) collects multiple arguments into a single array.
 - **Spread** (...) expands an array or object into individual elements or properties.
-

5. Does an arrow function have its own `arguments` object?

Answer:

No. Arrow functions do not have their own `arguments` object. Use rest parameters (...args) instead.

6. What is a pure function?

Answer:

A pure function always returns the same output for the same input and has no side effects.

7. What is an impure function?

Answer:

An impure function depends on or modifies external state, causing side effects.

8. What are first-class functions?

Answer:

Functions in JavaScript are first-class citizens, meaning they can be treated like any other value: assigned to variables, passed as arguments, returned from functions, and stored in data structures.

9. What is the purpose of `bind()`?

Answer:

`bind()` creates and returns a new function with its `this` value permanently set to a specified object.

10. Which functions are hoisted?

Answer:

Only **function declarations** are fully hoisted. Function expressions and arrow functions are not.

Practice Tasks

Task 1

Create a function declaration and call it **before** its declaration.

Observe the output.

Then repeat the same with:

- A function expression
- An arrow function

Observe the differences.

Task 2

Create a function `counter()` that uses a **closure** to keep track of how many times it has been called.

Expected output:

```
1
2
3
```

Task 3

Create an **IIFE** that prints:

```
Welcome to JavaScript
```

Task 4

Create a function `sum(...numbers)` that returns the sum of all numbers passed to it.

Test it with different numbers of arguments.

Task 5

Create two arrays:

```
const frontend = ["HTML", "CSS", "JavaScript"];  
const backend = ["Node.js", "Express"];
```

Use the **spread operator** to merge them into a new array and print the result.

 **Congratulations!** You have completed **Chapter 4: Functions**. This chapter covers one of the most important topics in JavaScript and provides a strong foundation for advanced concepts like arrays, asynchronous programming, DOM manipulation, and frameworks such as React.